Assignment 4: Weather Dashboard App

Overview

Build a weather dashboard that fetches live data from a public weather API based on a user-entered city name. This assignment focuses on asynchronous JavaScript, the Fetch API, and handling real-world network conditions (loading and errors).

Learning Objectives

- Make network requests with the Fetch API using async/await
- Parse and render JSON API responses dynamically
- Implement loading and error states in a user interface
- Handle real-world failure cases gracefully (bad input, network errors, rate limits)

Requirements

- A search input where the user enters a city name and submits a search
- Live data fetched from a public weather API (e.g. OpenWeatherMap, WeatherAPI, or instructor-approved equivalent)
- Display current temperature, conditions/description, a weather icon, and at least two additional data points (e.g. humidity, wind speed, feels-like)
- A visible loading indicator while a request is in progress
- A clear, user-facing error message for invalid city names or failed requests (no unhandled console errors)
- The layout is responsive and usable on both mobile and desktop widths

Technical Specifications

- Use async/await syntax (not raw .then() chains) for at least the primary fetch logic
- API keys must not be committed to the repository in plain text — use a .env file or a documented placeholder and add it to .gitignore
- Wrap fetch calls in try/catch and handle non-200 responses explicitly

Submission Guidelines

- Submit a GitHub repository link and a zipped project folder
- Include setup instructions for obtaining/using an API key in your README
- Include a screenshot in your submission showing the page with weather data of a city of your choice

Grading Rubric

Criterion	Points	Description
API integration & data fetching	25	Fetch requests work correctly and return usable data
Loading & error states	20	Loading indicator shown during requests; errors surfaced clearly to the user
Dynamic rendering of results	20	UI updates correctly based on the fetched data, not hardcoded values
UI design & responsiveness	15	Clean, legible layout that works across screen sizes
Code quality	20	Proper async/await usage, try/catch handling, sensible file structure, no exposed secrets
Total	**100**	

Note: Submissions that do not run, fail to load, or throw uncaught errors on open may receive a substantial deduction across all criteria, since graders cannot evaluate functionality that isn't demonstrable.